LAPORAN QUIZ MINGGU 9

PENGOLAHAN CITRA DIGITAL

202523430039

OLEH:

RACHEL MUTIARA HESA

24343110

PROGRAM STUDI INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRONIKA

FAKULTAS TEKNIK

UNIVERSITAS NEGERI PADANG

2026

1. Thresholding Techniques Comparison

- Kode program

Thresholding Techniques Comparison.py

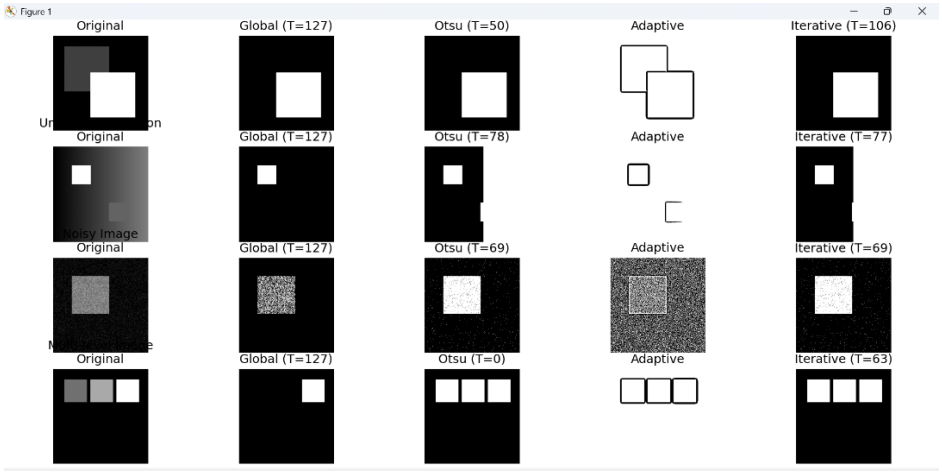
```python
import numpy as np
import cv2
import matplotlib.pyplot as plt
from scipy import ndimage

def praktikum_9_1():
    """
    Perbandingan teknik thresholding: Global, Otsu, dan Adaptive
    """
    print("PRAKTIKUM 9.1: PERBANDINGAN TEKNIK THRESHOLDING")
    print("=" * 60)

    # Buat citra test dengan berbagai karakteristik
    def create_test_images():
        images = {}

        # 1. Citra bimodal (ideal untuk thresholding)
        img_bimodal = np.zeros((256, 256), dtype=np.uint8)
        cv2.rectangle(img_bimodal, (30, 30), (150, 150), 50, -1)  # Dark object
        cv2.rectangle(img_bimodal, (100, 100), (220, 220), 200, -1)  # Bright object
        images['Bimodal Image'] = img_bimodal

        # 2. Citra dengan uneven illumination
        img_uneven = np.zeros((256, 256), dtype=np.uint8)
        # Create gradient background
        for i in range(256):
            img_uneven[:, i] = i // 2
        # Add objects
        cv2.rectangle(img_uneven, (50, 50), (100, 100), 255, -1)
        cv2.rectangle(img_uneven, (150, 150), (200, 200), 100, -1)
        images['Uneven Illumination'] = img_uneven

        # 3. Citra dengan noise
        img_noisy = np.zeros((256, 256), dtype=np.uint8)
        cv2.rectangle(img_noisy, (50, 50), (150, 150), 128, -1)
        # Add Gaussian noise
        noise = np.random.normal(0, 30, img_noisy.shape)
        img_noisy = np.clip(img_noisy.astype(float) + noise, 0, 255).astype(np.uint8)
        images['Noisy Image'] = img_noisy

        # 4. Citra dengan multiple intensity levels
        img_multi = np.zeros((256, 256), dtype=np.uint8)
        cv2.rectangle(img_multi, (30, 30), (90, 90), 80, -1)   # Dark gray
        cv2.rectangle(img_multi, (100, 30), (160, 90), 120, -1)  # Medium gray
        cv2.rectangle(img_multi, (170, 30), (230, 90), 180, -1)  # Light gray
        images['Multi-level Image'] = img_multi

        return images

    # Implementasi berbagai metode thresholding
    def apply_global_threshold(image, T=127):
        """Global thresholding"""
        _, binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
        return binary

    def apply_otsu_threshold(image):
        """Otsu's thresholding"""
        _, binary = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        return binary

    def apply_adaptive_threshold(image, block_size=11, C=2):
        """Adaptive thresholding"""
        if len(image.shape) == 3:
            image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        binary = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                       cv2.THRESH_BINARY, block_size, C)
        return binary

    def apply_iterative_threshold(image, max_iter=100, tolerance=1):
        """Iterative threshold selection"""
        # Initialize threshold
        T = np.mean(image)

        for i in range(max_iter):
            # Segment image
            foreground = image[image > T]
            background = image[image <= T]
```

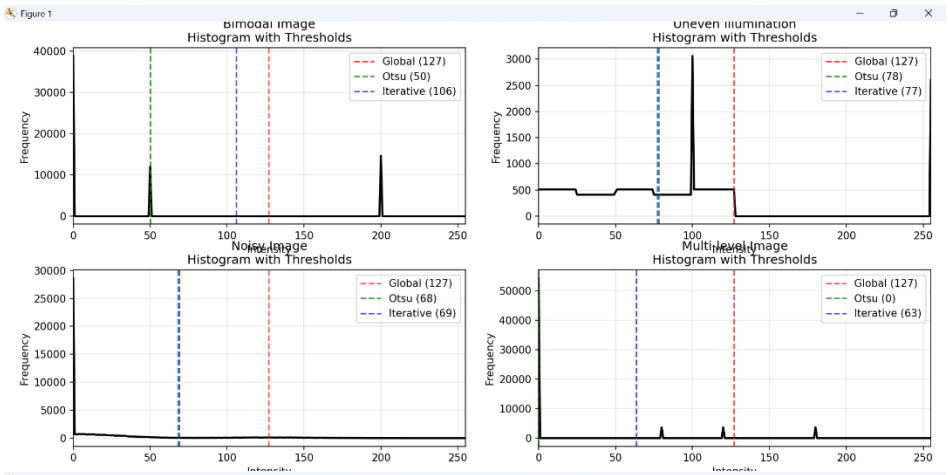
```
            # Compute means
            if len(foreground) > 0 and len(background) > 0:
                mu_fg = np.mean(foreground)
                mu_bg = np.mean(background)

                # New threshold
                T_new = (mu_fg + mu_bg) / 2

                # Check convergence
                if abs(T_new - T) < tolerance:
                    T = T_new
                    break

                T = T_new
            else:
                break

        # Apply threshold
        _, binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
        return binary, T

    # Buat citra test
    test_images = create_test_images()

    # Terapkan berbagai metode thresholding
    results = {}

    for name, image in test_images.items():
        # Convert to grayscale if needed
        if len(image.shape) == 3:
            gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        else:
            gray = image.copy()

        # Apply different thresholding methods
        global_binary = apply_global_threshold(gray, 127)
        otsu_binary = apply_otsu_threshold(gray)
        adaptive_binary = apply_adaptive_threshold(gray, 11, 2)
        iterative_binary, T_iter = apply_iterative_threshold(gray)

        # Get Otsu threshold value
        T_otsu, _ = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

        results[name] = {
            'original': gray,
            'global': global_binary,
            'otsu': otsu_binary,
            'adaptive': adaptive_binary,
            'iterative': iterative_binary,
            'T_otsu': T_otsu,
            'T_iter': T_iter[1] if isinstance(T_iter, tuple) else T_iter
        }

    # Visualisasi hasil
    fig, axes = plt.subplots(len(test_images), 5, figsize=(20, 4*len(test_images)))

    for idx, (name, result) in enumerate(results.items()):
        # Column 1: Original image + histogram
        axes[idx, 0].imshow(result['original'], cmap='gray')
        axes[idx, 0].set_title(f'{name}\nOriginal')
        axes[idx, 0].axis('off')

        # Column 2-5: Thresholding results
        methods = ['global', 'otsu', 'adaptive', 'iterative']
        titles = ['Global (T=127)', f'Otsu (T={result["T_otsu"]:.0f})',
                  'Adaptive', f'Iterative (T={result["T_iter"]:.0f})']

        for col, (method, title) in enumerate(zip(methods, titles), 1):
            axes[idx, col].imshow(result[method], cmap='gray')
            axes[idx, col].set_title(title)
            axes[idx, col].axis('off')

    plt.tight_layout()
    plt.show()
```

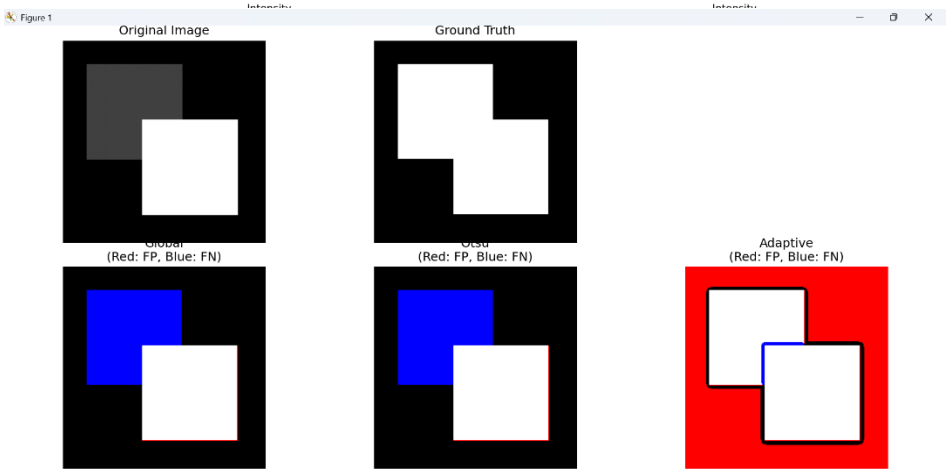
```python
    # Analisis histogram dan threshold selection
    print("\nANALISIS HISTOGRAM DAN THRESHOLD SELECTION")
    print("-" * 60)

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.ravel()

    for idx, (name, result) in enumerate(list(results.items())[:4]):
        # Plot histogram
        hist = cv2.calcHist([result['original']], [0], None, [256], [0, 256])
        axes[idx].plot(hist, 'k-', linewidth=2)

        # Add threshold lines
        axes[idx].axvline(x=127, color='r', linestyle='--', label='Global (127)', alpha=0.7)
        axes[idx].axvline(x=result['T_otsu'], color='g', linestyle='--',
                          label=f'Otsu ({result["T_otsu"]:.0f})', alpha=0.7)
        axes[idx].axvline(x=result['T_iter'], color='b', linestyle='--',
                          label=f'Iterative ({result["T_iter"]:.0f})', alpha=0.7)

        axes[idx].set_title(f'{name}\nHistogram with Thresholds')
        axes[idx].set_xlabel('Intensity')
        axes[idx].set_ylabel('Frequency')
        axes[idx].legend()
        axes[idx].grid(True, alpha=0.3)
        axes[idx].set_xlim([0, 255])

    plt.tight_layout()
    plt.show()

    # Evaluasi kuantitatif (simulasi ground truth)
    print("\nEVALUASI KUANTITATIF (DENGAN SIMULASI GROUND TRUTH)")
    print("-" * 70)

    # Buat ground truth untuk bimodal image
    gt_bimodal = np.zeros((256, 256), dtype=np.uint8)
    gt_bimodal[30:150, 30:150] = 1  # First object
    gt_bimodal[100:220, 100:220] = 1  # Second object

    # Hitung metrics untuk setiap metode
    def calculate_metrics(binary, ground_truth):
        """Calculate segmentation metrics"""
        # Ensure binary images
        binary = (binary > 0).astype(np.uint8)
        ground_truth = (ground_truth > 0).astype(np.uint8)

        # True Positive, False Positive, etc.
        tp = np.sum((binary == 1) & (ground_truth == 1))
        fp = np.sum((binary == 1) & (ground_truth == 0))
        fn = np.sum((binary == 0) & (ground_truth == 1))
        tn = np.sum((binary == 0) & (ground_truth == 0))

        # Calculate metrics
        accuracy = (tp + tn) / (tp + tn + fp + fn) if (tp+tn+fp+fn) > 0 else 0
        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
        iou = tp / (tp + fp + fn) if (tp + fp + fn) > 0 else 0

        return {
            'accuracy': accuracy,
            'precision': precision,
            'recall': recall,
            'f1_score': f1_score,
            'iou': iou
        }

    # Evaluasi untuk bimodal image
    bimodal_result = results['Bimodal Image']

    print(f"{'Method':<15} {'Accuracy':<10} {'Precision':<10} {'Recall':<10} {'F1-Score':<10} {'IoU':<10}")
    print("-" * 70)

    methods = ['global', 'otsu', 'adaptive', 'iterative']
    method_names = ['Global', 'Otsu', 'Adaptive', 'Iterative']

    for method, method_name in zip(methods, method_names):
        metrics = calculate_metrics(bimodal_result[method], gt_bimodal)
        print(f"{method_name:<15} {metrics['accuracy']:<10.3f} {metrics['precision']:<10.3f} "
              f"{metrics['recall']:<10.3f} {metrics['f1_score']:<10.3f} {metrics['iou']:<10.3f}")

    # Visual comparison dengan ground truth
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    # Row 1: Original and ground truth
    axes[0, 0].imshow(bimodal_result['original'], cmap='gray')
    axes[0, 0].set_title('Original Image')
    axes[0, 0].axis('off')

    axes[0, 1].imshow(gt_bimodal, cmap='gray')
    axes[0, 1].set_title('Ground Truth')
    axes[0, 1].axis('off')
```

```python
    # Row 2: Thresholding results dengan overlay errors
    for idx, (method, method_name) in enumerate(zip(methods[:3], method_names[:3])):
        result_binary = (bimodal_result[method] > 0).astype(np.uint8)

        # Create error visualization
        error_image = np.zeros((256, 256, 3), dtype=np.uint8)

        # True Positive: White
        tp_mask = (result_binary == 1) & (gt_bimodal == 1)
        error_image[tp_mask] = [255, 255, 255]

        # False Positive: Red (segmented but not in GT)
        fp_mask = (result_binary == 1) & (gt_bimodal == 0)
        error_image[fp_mask] = [255, 0, 0]

        # False Negative: Blue (in GT but not segmented)
        fn_mask = (result_binary == 0) & (gt_bimodal == 1)
        error_image[fn_mask] = [0, 0, 255]

        axes[1, idx].imshow(error_image)
        axes[1, idx].set_title(f'{method_name}\n(Red: FP, Blue: FN)')
        axes[1, idx].axis('off')

    plt.tight_layout()
    plt.show()

    # Kesimpulan dan rekomendasi
    print("\nKESIMPULAN DAN REKOMENDASI")
    print("-" * 40)
    print("""
1. Global Thresholding:
   - Cocok untuk citra dengan kontras tinggi dan illumination uniform
   - Sensitif terhadap uneven illumination dan noise
   - Butuh manual threshold selection

2. Otsu's Method:
   - Fully automatic
   - Optimal untuk citra bimodal histogram
   - Tidak efektif untuk multi-modal atau uneven illumination

3. Adaptive Thresholding:
   - Cocok untuk uneven illumination
   - Robust terhadap variasi intensitas lokal
   - Parameter block size dan C perlu tuning

4. Iterative Thresholding:
   - Self-tuning threshold
   - Cocok untuk citra dengan distribusi intensity yang jelas
   - Computationally lebih expensive
    """)

    return results

thresholding_results = praktikum_9_1()
```

- Output








```
PRAKTIKUM 9.1: PERBANDINGAN TEKNIK THRESHOLDING
================================================================

ANALISIS HISTOGRAM DAN THRESHOLD SELECTION
----------------------------------------------------------------

EVALUASI KUANTITATIF (DENGAN SIMULASI GROUND TRUTH)
--------------------------------------------------------------------------------
Method          Accuracy    Precision   Recall      F1-Score    IoU
--------------------------------------------------------------------------------
Global          0.815       0.984       0.548       0.703       0.543
Otsu            0.815       0.984       0.548       0.703       0.543
Adaptive        0.448       0.420       0.985       0.589       0.417
Iterative       0.815       0.984       0.548       0.703       0.543

KESIMPULAN DAN REKOMENDASI
----------------------------------------

1. Global Thresholding:
   - Cocok untuk citra dengan kontras tinggi dan illumination uniform
   - Sensitif terhadap uneven illumination dan noise
   - Butuh manual threshold selection

2. Otsu's Method:
   - Fully automatic
   - Optimal untuk citra bimodal histogram
   - Tidak efektif untuk multi-modal atau uneven illumination

3. Adaptive Thresholding:
   - Cocok untuk uneven illumination
   - Robust terhadap variasi intensitas lokal
   - Parameter block size dan C perlu tuning

4. Iterative Thresholding:
   - Self-tuning threshold
   - Cocok untuk citra dengan distribusi intensity yang jelas
   - Computationally lebih expensive
```

- Penjelasan

Program praktikum_9_1() bertujuan untuk membandingkan beberapa metode thresholding dalam proses segmentasi citra, yaitu Global Thresholding, Otsu, Adaptive, dan Iterative. Program ini terlebih dahulu membuat beberapa citra uji dengan karakteristik berbeda seperti kontras tinggi, pencahayaan tidak merata, adanya noise, dan variasi intensitas. Selanjutnya, setiap metode diterapkan pada citra tersebut dan hasilnya divisualisasikan untuk dibandingkan. Program juga menganalisis histogram untuk melihat pemilihan nilai threshold serta melakukan evaluasi

kuantitatif menggunakan metrik seperti accuracy, precision, recall, F1-score, dan IoU dengan bantuan ground truth. Dengan demikian, program ini membantu memahami kelebihan dan kekurangan masing-masing metode thresholding dalam berbagai kondisi citra.

2. Edge Detection dan Region-based Segmentation

- Kode program

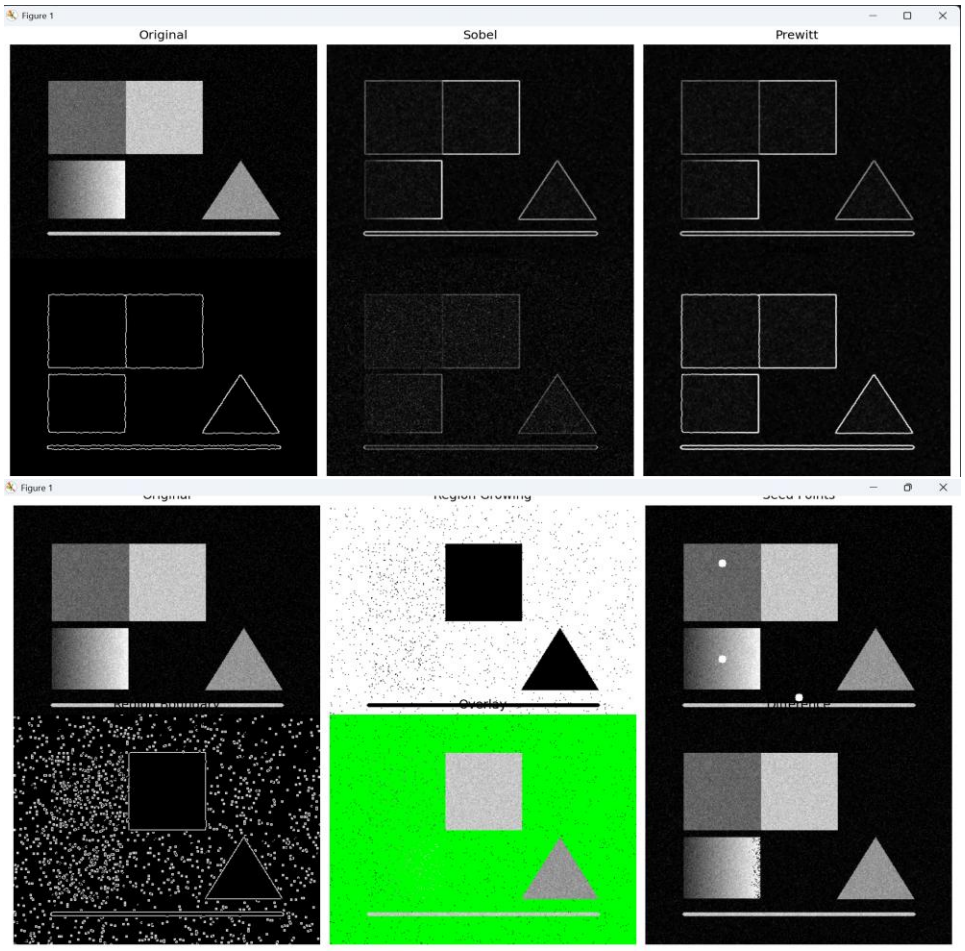
Edge Detection dan Region-based Segmentation.py

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

def praktikum_9_2():
    print("\nPRAKTIKUM 9.2: EDGE DETECTION DAN REGION-BASED SEGMENTATION")
    print("=" * 70)

    # ------------------------
    # BUAT GAMBAR TEST
    # ------------------------
    def create_edge_test_image():
        img = np.zeros((300, 400), dtype=np.uint8)

        cv2.rectangle(img, (50, 50), (150, 150), 100, -1)
        cv2.rectangle(img, (151, 50), (250, 150), 200, -1)

        for i in range(50, 150):
            img[160:240, i] = 50 + (i - 50) * 2

        triangle_cnt = np.array([(300, 160), (350, 240), (250, 240)])
        cv2.drawContours(img, [triangle_cnt], 0, 150, -1)

        cv2.line(img, (50, 260), (350, 260), 200, 3)

        noise = np.random.normal(0, 15, img.shape)
        img = np.clip(img + noise, 0, 255).astype(np.uint8)

        return img

    # ------------------------
    # EDGE DETECTION
    # ------------------------
    def sobel(image):
        sx = cv2.Sobel(image, cv2.CV_64F, 1, 0)
        sy = cv2.Sobel(image, cv2.CV_64F, 0, 1)
        mag = np.sqrt(sx**2 + sy**2)
        mag = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
        return mag.astype(np.uint8)

    def prewitt(image):
        kx = np.array([[1,1,1],[0,0,0],[-1,-1,-1]])
        ky = np.array([[-1,0,1],[-1,0,1],[-1,0,1]])
        px = cv2.filter2D(image.astype(float), -1, kx)
        py = cv2.filter2D(image.astype(float), -1, ky)
        mag = np.sqrt(px**2 + py**2)
        mag = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
        return mag.astype(np.uint8)

    def canny(image):
        blur = cv2.GaussianBlur(image, (5,5), 1.4)
        return cv2.Canny(blur, 50, 150)

    def laplacian(image):
        lap = cv2.Laplacian(image, cv2.CV_64F)
        lap = np.abs(lap)
        lap = cv2.normalize(lap, None, 0, 255, cv2.NORM_MINMAX)
        return lap.astype(np.uint8)

    # ------------------------
    # REGION GROWING (AMAN)
    # ------------------------
    def region_growing(image, seeds, threshold=25):
        h, w = image.shape
        visited = np.zeros((h, w), dtype=bool)
        result = np.zeros_like(image)

        for seed in seeds:
            stack = [seed]

            while stack:
                x, y = stack.pop()

                if not (0 <= x < h and 0 <= y < w):
                    continue
                if visited[x, y]:
                    continue
```

```
                visited[x, y] = True
                result[x, y] = 255

                for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nx, ny = x+dx, y+dy
                    if 0 <= nx < h and 0 <= ny < w:
                        if not visited[nx, ny]:
                            if abs(int(image[nx, ny]) - int(image[x, y])) < threshold:
                                stack.append((nx, ny))

        return result

    # ------------------------
    # PROSES
    # ------------------------
    img = create_edge_test_image()

    sob = sobel(img)
    pre = prewitt(img)
    can = canny(img)
    lap = laplacian(img)

    seeds = [(75,100),(200,100),(250,200)]
    rg = region_growing(img, seeds)

    # ------------------------
    # 1. EDGE DETECTION (6 GAMBAR)
    # ------------------------
    fig1, ax = plt.subplots(2,3, figsize=(15,10))

    ax[0,0].imshow(img, cmap='gray')
    ax[0,0].set_title("Original")

    ax[0,1].imshow(sob, cmap='gray')
    ax[0,1].set_title("Sobel")

    ax[0,2].imshow(pre, cmap='gray')
    ax[0,2].set_title("Prewitt")

    ax[1,0].imshow(can, cmap='gray')
    ax[1,0].set_title("Canny")

    ax[1,1].imshow(lap, cmap='gray')
    ax[1,1].set_title("Laplacian")

    combined = np.maximum(sob, can)
    ax[1,2].imshow(combined, cmap='gray')
    ax[1,2].set_title("Combined")

    for a in ax.ravel():
        a.axis('off')

    plt.tight_layout()
    plt.show(block=True)

    # ------------------------
    # 2. REGION SEGMENTATION (6 GAMBAR)
    # ------------------------
    fig2, ax = plt.subplots(2,3, figsize=(15,10))

    ax[0,0].imshow(img, cmap='gray')
    ax[0,0].set_title("Original")

    ax[0,1].imshow(rg, cmap='gray')
    ax[0,1].set_title("Region Growing")

    seed_img = img.copy()
    for s in seeds:
        cv2.circle(seed_img, (s[1], s[0]), 5, 255, -1)

    ax[0,2].imshow(seed_img, cmap='gray')
    ax[0,2].set_title("Seed Points")

    ax[1,0].imshow(cv2.Canny(rg,50,150), cmap='gray')
    ax[1,0].set_title("Region Boundary")
```

```
    overlay = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    overlay[rg>0] = [0,255,0]
    ax[1,1].imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB))
    ax[1,1].set_title("Overlay")

    ax[1,2].imshow(np.abs(img - rg), cmap='gray')
    ax[1,2].set_title("Difference")

    for a in ax.ravel():
        a.axis('off')

    plt.tight_layout()
    plt.show(block=True)


    return img, sob, pre, can, lap, rg


# ------------------------
# JALANKAN
# ------------------------
praktikum_9_2()
```

- Output



- Penjelasan

Program praktikum bertujuan untuk mendemonstrasikan dan membandingkan metode edge detection dan region-based segmentation pada citra. Program ini dimulai dengan membuat gambar uji yang memiliki berbagai bentuk, gradien, dan noise untuk mensimulasikan kondisi nyata. Selanjutnya, beberapa metode deteksi tepi seperti Sobel, Prewitt, Canny, dan Laplacian diterapkan untuk mengekstraksi batas objek. Selain itu, digunakan metode region growing untuk segmentasi berbasis area dengan titik awal (seed). Hasil dari setiap metode kemudian divisualisasikan dalam bentuk beberapa gambar, termasuk kombinasi edge, batas region, overlay, dan perbedaan citra. Dengan demikian, program ini membantu memahami bagaimana berbagai metode bekerja dalam mendeteksi tepi dan memisahkan objek berdasarkan wilayah.

Link github: https://github.com/RachelMutiaraHesa/PengolahanCitraDigital.git